جلسهٔ ۱۳ — شبکه‌های عصبیِ کانولوشنی (Convolutional Neural Networks)

جزوهٔ درس — هفتهٔ سیزدهم

نگاهِ کلی به این جلسه

این جلسه اوجِ بخشِ یادگیریِ درس است. در هفتهٔ ۳، ما کرنل‌ها را با دست طراحی کردیم (بلور، سوبل) و اعدادشان را خودمان انتخاب کردیم. در هفتهٔ ۱۲ دیدیم که ویژگی‌های دست‌طراحی (HOG) به سقف می‌خورند. امروز ایدهٔ بزرگ را می‌آموزیم: بگذاریم شبکه **خودِ کرنل‌ها را از داده یاد بگیرد** — و هم‌زمان طبقه‌بند را هم یاد بگیرد. این همان **شبکهٔ عصبیِ کانولوشنی** (CNN) است، ستون‌فقراتِ بینایی ماشینِ مدرن.

یادداشت

نتیجهٔ واقعیِ امروز: یک CNN کوچک، ارقامِ ۰ تا ۹ را با دقتِ **۹۷٪** می‌خواند. این دقیقاً همان «خوانندهٔ ارقامِ کنتور» است که DocScan به آن نیاز دارد — و آن را از صفر آموزش می‌دهیم. جالب اینکه روی همین ارقامِ ساده، یک SVM کلاسیک هم ۹۹٪ می‌گیرد؛ این تضاد، درسِ صادقانه‌ای دربارهٔ «کِی CNN واقعاً برنده است» به ما می‌دهد.

یک تشبیهِ ساده

تصور کنید یک کارآموزِ کنترلِ کیفیت می‌خواهد قطعاتِ سالم را از معیوب تشخیص دهد. در روشِ کلاسیک، سرپرست به او فهرستی از قواعد می‌دهد: «این خراش را ببین، آن لبه را چک کن». در روشِ CNN، به کارآموز هزاران قطعهٔ برچسب‌دار نشان می‌دهند و او خودش کم‌کم یاد می‌گیرد به چه نشانه‌هایی نگاه کند — و این نشانه‌ها را کسی از پیش به او نگفته. جالب‌تر اینکه او ابتدا نشانه‌های ساده (لبه، رنگ) را یاد می‌گیرد و بعد آن‌ها را در قالبِ الگوهای پیچیده‌تر ترکیب می‌کند. CNN دقیقاً همین‌گونه است: سلسله‌ای از لایه‌ها که هر کدام الگوی پیچیده‌تری از لایهٔ قبل می‌سازد، و همهٔ این «یادگرفتنِ نشانه‌ها» از دادهٔ برچسب‌دار می‌آید.

۱ از کرنل تا فیلترِ آموخته‌شده

۱.۱ کانولوشن: شما از قبل بلدید

در هفتهٔ ۳ یاد گرفتیم: یک کرنلِ کوچک را روی تصویر می‌لغزانیم، ضرب‌ودرجمع می‌کنیم و یک تصویرِ فیلترشده می‌گیریم. کرنلِ سوبل لبه‌ها را پیدا می‌کند و کرنلِ بلور صاف می‌کند. ما اعداد را انتخاب می‌کردیم.

یک **لایهٔ کانولوشنی** دقیقاً همین عملیات است — با این تفاوتِ حیاتی که اعدادِ کرنل **آموخته** می‌شوند، نه دستی انتخاب.

نکتهٔ کلیدی

ساده‌ترین تعریفِ CNN: «کانولوشن با کرنل‌هایی که با آموزش انتخاب می‌شوند، نه با دست». همهٔ پیچیدگیِ ظاهری از همین ایدهٔ ساده می‌آید.

۲.۱ شبکه چه چیزی یاد می‌گیرد؟

پس از آموزشِ CNN ارقامِ ما، هشت فیلترِ ۵ × ۵ لایهٔ اول را که شبکه یاد گرفته استخراج کردیم:



۸ فیلترِ لایهٔ اول که CNN برای خواندنِ ارقام یاد گرفت — هیچ‌کس آن‌ها را طراحی نکرد

هیچ‌کس این اعداد را انتخاب نکرد؛ گرادیان کاهشی (gradient descent) آشکارسازهای لبه/خطِ مفید برای تشخیصِ ارقام را پیدا کرد. لایه‌های اولیه لبه‌ها را یاد می‌گیرند، لایه‌های عمیق‌تر آن‌ها را در قالبِ اجزا و سپس شکل‌های کامل ترکیب می‌کنند.

۳.۱ یک مثالِ عددیِ کانولوشن (یادآوریِ هفتهٔ ۳)

بیایید مطمئن شویم کانولوشن را دقیق به یاد داریم. فرض کنید یک تکه از تصویر و یک کرنلِ ۳ × ۳ داریم:

ریاضیات

$$\text{کرنلِ لبهٔ عمودی} = \begin{pmatrix} -۱ & ۰ & ۱ \\ -۱ & ۰ & ۱ \\ -۱ & ۰ & ۱ \end{pmatrix}, \qquad \text{تکهٔ تصویر} = \begin{pmatrix} ۲ & ۰ & ۱ \\ ۴ & ۶ & ۳ \\ ۱ & ۲ & ۵ \end{pmatrix}$$

خروجیِ کانولوشن در مرکز = مجموعِ ضربِ عنصربه‌عنصر:

$$(۲)(-۱) + (۰)(۰) + (۱)(۱) + (۴)(-۱) + (۶)(۰) + (۳)(۱) + (۱)(-۱) + (۲)(۰) + (۵)(۱)$$

$$= (-۲ + ۰ + ۱) + (-۴ + ۰ + ۳) + (-۱ + ۰ + ۵) = -۱ - ۱ + ۴ = ۲$$

این عددِ ۲ می‌گوید «در این نقطه یک لبهٔ عمودیِ ملایم (روشن‌تر در سمتِ راست) وجود دارد». در هفتهٔ ۳ ما این کرنلِ $[-۱, ۰, ۱]$ را انتخاب کردیم. در CNN، شبکه با آموزش کشف می‌کند که چنین کرنلی مفید است و اعدادش را خودش تنظیم می‌کند. تفاوت فقط در «چه کسی اعداد را انتخاب می‌کند» است.

نکتهٔ کلیدی

این پیوند حیاتی است: CNN چیزِ کاملاً جدیدی اختراع نکرده؛ همان کانولوشنِ آشنا را گرفته و فقط انتخابِ کرنل را از انسان به الگوریتمِ آموزش سپرده است.

۴.۱ نقشهٔ ویژگی

اعمالِ یک فیلترِ آموخته‌شده روی تصویر، یک **نقشهٔ ویژگی** (feature map) می‌دهد: نقشه‌ای که نشان می‌دهد آن الگو کجای تصویر ظاهر شده. یک لایه چندین فیلتر دارد، پس چندین نقشهٔ ویژگی تولید می‌کند — یک «پشته» از آشکارسازهای الگو. لایهٔ بعدی روی این پشته کانولوشن می‌زند و الگوهای ساده را به الگوهای پیچیده تبدیل می‌کند.

۲ کالبدشکافیِ یک CNN

یک CNN از چند نوع لایه ساخته می‌شود:

لایه	کارش	چرا
کانولوشن	فیلترهای آموخته → نقشه‌های ویژگی	کشفِ الگو
ReLU	$\max(۰, x)$ روی هر عنصر	افزودنِ غیرخطی‌بودن
Pooling	کوچک‌سازی (مثلاً بیشینهٔ ۲ × ۲)	کوچک‌سازی + ناوردایی
تمام‌متصل (FC)	تخت‌کردن → امتیازِ کلاس‌ها	تصمیمِ نهایی
Softmax	امتیاز → احتمال	انتخابِ کلاس

۱.۲ شمردنِ پارامترها

بیایید بشماریم شبکهٔ کوچکِ ما چند پارامتر دارد — این عادتِ خوبی برای فهمِ اندازهٔ مدل است:

ریاضیات

- لایهٔ conv۱: ۸ فیلترِ ۳ × ۳ روی ۱ کانالِ ورودی ⟹ پارامتر $= ۸ \times (۳ \times ۳ \times ۱) + ۸ = ۸۰$ بایاس.
- لایهٔ conv۲: ۱۶ فیلترِ ۳ × ۳ روی ۸ کانال ⟹ پارامتر $= ۱۶ \times (۳ \times ۳ \times ۸) + ۱۶ = ۱۱۶۸$.
- لایهٔ fc: از ۶۴ ورودی به ۱۰ خروجی ⟹ پارامتر $= ۶۴ \times ۱۰ + ۱۰ = ۶۵۰$.
- **مجموع** ≈ ۱۸۹۸ پارامتر.

نکتهٔ کلیدی

کمتر از ۲۰۰۰ پارامتر — بسیار کوچک! برای مقایسه، AlexNet حدودِ ۶۰ میلیون و ResNet-۵۰ حدودِ ۲۵ میلیون پارامتر دارد. همین شبکهٔ ریز، ارقام را با ۹۷٪ می‌خواند، چون مسئله ساده است و معماریِ کانولوشنی برای تصویر بسیار کارآمد است.

۲.۲ چرا ReLU (غیرخطی‌بودن)؟

اگر فقط کانولوشن‌ها را روی هم بگذاریم، حاصل هنوز یک فیلترِ خطیِ بزرگ است — هیچ قدرتِ اضافه‌ای نمی‌گیریم.

ریاضیات

ترکیبِ عملیاتِ خطی باز هم خطی است: اگر $f(x) = Ax$ و $g(x) = Bx$، آن‌گاه $g(f(x)) = BAx$ که باز هم یک تبدیلِ خطیِ واحد است. برای رهایی از این دام، بینِ لایه‌ها یک تابعِ غیرخطی می‌گذاریم. **ReLU** یعنی $\mathrm{ReLU}(x) = \max(0, x)$: هر مقدارِ منفی را صفر می‌کند و مثبت‌ها را دست‌نخورده می‌گذارد.

نکتهٔ کلیدی

بدونِ غیرخطی‌بودن، ده لایهٔ کانولوشن به یک لایه فرومی‌کاهند. ReLU همان چیزی است که واژهٔ «عمیق» را معنادار می‌کند.

۳.۲ چرا Pooling؟

بیشینه‌گیری (max-pooling) در هر پنجرهٔ کوچک، قوی‌ترین پاسخ را نگه می‌دارد و اندازه را نصف می‌کند. سه فایده:

- **تحملِ جابه‌جایی:** رقمی که یک پیکسل جابه‌جا شده، باز همان واحدها را فعال می‌کند.
- **کاهشِ محاسبات:** تصویرِ کوچک‌تر یعنی پردازشِ سریع‌تر.
- **رشدِ میدانِ دید:** لایه‌های بعدی ناحیهٔ بزرگ‌تری از تصویرِ اصلی را «می‌بینند».

یادداشت

شهودِ pooling: «این الگو را جایی در این ناحیه دیدم»، نه «دقیقاً در این پیکسل». همین کمی ابهامِ مکانی، شبکه را در برابرِ جابه‌جاییِ کوچک مقاوم می‌کند.

۴.۲ Softmax با یک مثالِ عددی

لایهٔ آخر ۱۰ عددِ خام (logit) می‌دهد که می‌توانند هر مقداری باشند. **Softmax** آن‌ها را به احتمال‌هایی تبدیل می‌کند که جمعشان ۱ می‌شود.

ریاضیات

برای امتیازهای خام z، احتمالِ کلاسِ i:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

مثال: اگر سه امتیازِ خام $z = [۲٫۰,\ ۱٫۰,\ ۰٫۱]$ باشند:

$$e^{۲٫۰} = ۷٫۳۹, \quad e^{۱٫۰} = ۲٫۷۲, \quad e^{۰٫۱} = ۱٫۱۱, \quad \text{مجموع} = ۱۱٫۲۲$$

$$p = [۰٫۶۶,\ ۰٫۲۴,\ ۰٫۱۰]$$

پس شبکه با احتمالِ ۶۶٪ کلاسِ اول را پیش‌بینی می‌کند. توجه کنید که softmax اختلاف‌ها را تشدید می‌کند (نمایی است) و همیشه خروجیِ مثبت با جمعِ ۱ می‌دهد.

نکتهٔ کلیدی

چرا نمایی؟ چون هم همه‌چیز را مثبت می‌کند و هم به امتیازهای بزرگ‌تر وزنِ به‌مراتب بیشتری می‌دهد — که رفتارِ «انتخابِ برنده» را مدل می‌کند. سپس آنتروپیِ متقاطع روی همین احتمال‌ها زیان را حساب می‌کند.

۵.۲ CNN ارقامِ ما، سرتاسر

- ورودی: رقمِ $\times$ $\rightarrow$ کانولوشن (۸ فیلتر)+ReLU+Pool $\rightarrow$ $\times$.
- $\rightarrow$ کانولوشن (۱۶ فیلتر)+ReLU+Pool $\rightarrow$ $\times$ $\rightarrow$ تخت‌سازی (۶۴) $\rightarrow$ FC $\rightarrow$ ۱۰ امتیاز.
- Softmax ده امتیاز را به احتمال تبدیل می‌کند؛ بزرگ‌ترین احتمال، رقمِ پیش‌بینی‌شده است.

دو بلوکِ کانولوشن و یک لایهٔ خطی — با معیارهای امروزی بسیار کوچک، اما همین ارقام را با ۹۷٪ می‌خواند.

۳ آموزشِ یک CNN

شبکه چطور فیلترها را یاد می‌گیرد؟ چهار گامِ تکرارشونده:

۱. **گذرِ روبه‌جلو (forward):** تصویر را از شبکه عبور بده و احتمال‌های پیش‌بینی‌شده را بگیر.

۲. **زیان (loss):** اندازه بگیر چقدر اشتباه است (آنتروپیِ متقاطع بینِ پیش‌بینی و برچسبِ واقعی).

۳. **پس‌انتشار (backprop):** حساب کن هر وزن چقدر در زیان اثر داشت (گرادیان).

۴. **به‌روزرسانی (update):** هر وزن را کمی در جهتِ کاهشِ زیان تغییر بده (گرادیان کاهشی).

این چرخه روی هزاران تصویر و ده‌ها epoch تکرار می‌شود و فیلترها به‌تدریج به آشکارسازهای خوب تبدیل می‌شوند.

۱.۳ زیانِ آنتروپیِ متقاطع

ریاضیات

شبکه برای هر کلاس یک احتمال می‌دهد. زیانِ آنتروپیِ متقاطع برای یک نمونه:

$$L = -\log\left(p_{\text{کلاسِ درست}}\right)$$

اگر شبکه به کلاسِ درست احتمالِ بالا بدهد، $p \to ۱$ و $L \to ۰$. اگر با اطمینان اشتباه کند ($p \to ۰$)، زیان به‌شدت بزرگ می‌شود. آموزش، میانگینِ این زیان را روی کلِ داده کمینه می‌کند.

۲.۳ گرادیان کاهشی، شهودی

پس‌انتشار و گرادیان کاهشی قلبِ آموزش‌اند. شهودِ ساده: تصور کنید در مهی غلیظ روی یک تپه ایستاده‌اید و می‌خواهید به پایین‌ترین نقطهٔ دره برسید.

ریاضیات

شما شیبِ زمین را زیرِ پایتان حس می‌کنید (این همان گرادیان است) و یک گامِ کوچک در جهتِ سراشیبی برمی‌دارید. اندازهٔ گام همان نرخِ یادگیری است. با تکرار، به کفِ دره (کمینهٔ زیان) نزدیک می‌شوید:

$$w_{\text{جدید}} = w_{\text{قدیم}} - \eta \cdot \frac{\partial L}{\partial w}$$

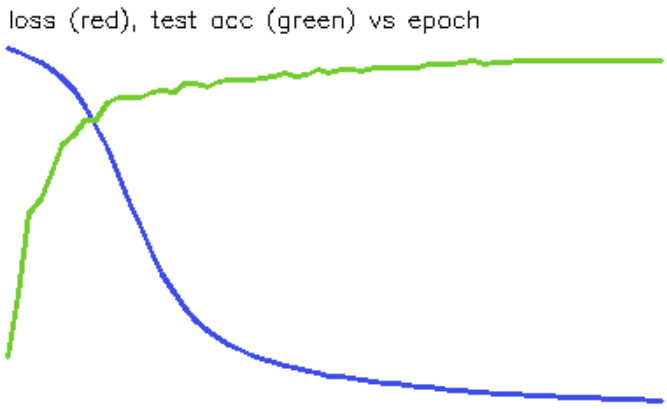
که η نرخِ یادگیری و $\frac{\partial L}{\partial w}$ گرادیانِ زیان نسبت به وزن است. پس‌انتشار (backprop) روشی کارآمد برای محاسبهٔ این گرادیان برای همهٔ وزن‌ها به‌طورِ هم‌زمان است.

یادداشت

گامِ خیلی بزرگ (η بزرگ) ممکن است از روی دره بپرید و واگرا شوید؛ گامِ خیلی کوچک، رسیدن به کف را بی‌نهایت کند می‌کند. یافتنِ η مناسب، مهم‌ترین تنظیمِ عملیِ آموزش است.

۳.۳ آموزش در عمل

منحنیِ آموزشِ واقعیِ CNN ما:



قرمز: زیانِ آموزش (کاهشی) سبز: دقتِ آزمون (صعودی تا نزدیکِ ۹۷٫۰)

خطِ قرمز (زیان) هر epoch پایین می‌آید و خطِ سبز (دقتِ آزمون) بالا می‌رود و سپس به سقف می‌رسد. تماشای هم‌زمانِ این دو منحنی، راهِ زندهٔ تشخیصِ کم‌برازش/بیش‌برازش است.

۴.۳ دسته‌های کوچک، epoch، نرخِ یادگیری

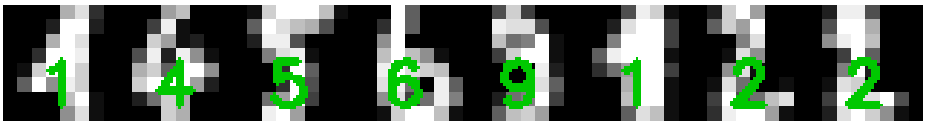
- **دستهٔ کوچک (mini-batch):** به‌جای کلِ داده، هر بار روی گروهی کوچک (ما ۱۲۸ تصویر) به‌روزرسانی می‌کنیم — سریع‌تر و نرم‌تر.
- **epoch:** یک گذرِ کامل روی کلِ دادهٔ آموزش. ما ۶۰ تا اجرا کردیم.
- **نرخِ یادگیری (learning rate):** اندازهٔ هر گامِ به‌روزرسانی. خیلی بزرگ → واگرایی؛ خیلی کوچک → کندیِ مفرط.

هشدار / دام

بیشترِ شکست‌های آموزش از نرخِ یادگیریِ بد می‌آید. اگر زیان منفجر شد یا اصلاً حرکت نکرد، اولین چیزی که تغییر می‌دهید نرخِ یادگیری است.

۴ نتایج

۱.۴ خواندنِ ارقام — دستاوردِ DocScan



ارقامِ آزمون با پیش‌بینیِ CNN (سبز = درست). این همان خوانندهٔ ارقامِ کنتور است

۲.۴ CNN در برابرِ کلاسیک — مقایسهٔ صادقانه

روش	دقتِ آزمون (ارقام)
CNN (فیلترهای آموخته)	۹۷٫۰
SVM روی پیکسلِ خام	۹۹٫۰

نکتهٔ صادقانه: روی ارقامِ ساده و تمیز 28×28، یادگیریِ ماشینِ کلاسیک از قبل عالی است — CNN اینجا معجزه نمی‌کند. برتریِ واقعیِ CNN روی تصاویرِ سخت، بزرگ و متنوع ظاهر می‌شود، و به‌ویژه با **یادگیریِ انتقالی** (هفتهٔ ۱۴).

۳.۴ اشتراکِ وزن: چرا CNN از شبکهٔ کاملاً‌متصل بهتر است

چرا اصلاً کانولوشن؟ چرا مثلِ شبکه‌های عصبیِ معمولی همهٔ پیکسل‌ها را به همهٔ نورون‌ها وصل نکنیم؟ دو دلیلِ قدرتمند:

ریاضیات

۱) اشتراکِ وزن (weight sharing): یک کرنلِ 5×5 فقط ۲۵ وزن دارد که روی کلِ تصویر لغزانده می‌شود. یک لایهٔ کاملاً‌متصل برای تصویرِ 28×28 به میلیون‌ها وزن نیاز دارد. کانولوشن با ۲۵ وزن همان کار را می‌کند — کارآمدیِ عظیم.

۲) ناوردایی به مکان: چون همان کرنل همه‌جا اعمال می‌شود، اگر یک رقم در گوشهٔ چپ یا راستِ تصویر باشد، با همان فیلتر شناسایی می‌شود. شبکهٔ کاملاً‌متصل باید این را جداگانه برای هر مکان یاد بگیرد.

یادداشت

این دو ویژگی — پارامترهای کم و ناوردایی به مکان — دلیلِ اصلیِ سلطهٔ CNN بر تصویر است. آن‌ها «سوگیریِ استقرایی» (inductive bias) مناسبِ داده‌های تصویری را در معماری تعبیه می‌کنند: می‌دانیم الگوهای مفید در تصویر محلی‌اند و می‌توانند هرجا ظاهر شوند.

۴.۴ CNN کِی واقعاً برنده است؟

- مجموعه‌داده‌های بزرگ و متنوع که ویژگیِ دستی نمی‌تواند تنوعشان را بگیرد.
- مسائلی که ویژگیِ مفید در آن‌ها ناشناخته است (شبکه خودش کشفشان می‌کند).
- یادآوریِ هفتهٔ ۱۲: HOG+SVM روی خودروهای شبیه‌به‌هم روی ۴۸٪ متوقف شد — دقیقاً همان‌جا که ویژگیِ آموخته کمک می‌کند.

نکتهٔ کلیدی

قاعدهٔ سرانگشتی: مسئلهٔ کوچک و تمیز → کلاسیک کافی است. مسئلهٔ بزرگ و شلوغ → یادگیریِ عمیق جلو می‌افتد. یک مهندسِ خوب می‌داند کدام است.

۵ ژرف‌نگری

۱.۵ سلسله‌مراتبِ ویژگی

- لایهٔ ۱: لبه‌ها و خطوطِ ساده (مثلِ فیلترهای آموخته‌ای که دیدیم).
- لایهٔ ۲ و ۳: گوشه‌ها، منحنی‌ها، اجزای رقم (حلقه‌ها، محلِ اتصال).
- لایه‌های عمیق‌تر: آشکارسازهای شکلِ کامل که برای یک رقمِ خاص فعال می‌شوند.

CNN پیچیدگی را تدریجی می‌سازد: اجزای ساده در قالبِ کل‌های معنادار ترکیب می‌شوند، و همهٔ این‌ها آموخته می‌شوند.

۲.۵ منظم‌سازی: صادق نگه‌داشتنِ CNN

هاCNN پارامترهای زیادی دارند، پس می‌توانند بیش‌برازش کنند (درسِ هفتهٔ ۱۲ بازمی‌گردد). ابزارهای مهار:

- **Dropout:** در حینِ آموزش، به‌طورِ تصادفی برخی فعال‌سازی‌ها را صفر کن تا شبکه به هیچ واحدِ خاصی وابسته نشود.
- **افزایشِ داده (augmentation):** چرخش/جابه‌جاییِ تصادفیِ تصاویرِ آموزش — دادهٔ اضافیِ رایگان و ناوردایی بیشتر.
- **کاهشِ وزن (weight decay):** جریمهٔ وزن‌های بزرگ برای سادّه‌نگه‌داشتنِ مدل.

۳.۵ معماری‌های معروف: یک نگاهِ کوتاه

CNN ما کوچک است، اما در دلِ همان خانواده‌ای است که تاریخِ بینایی ماشین را ساخت:

- **LeNet-۵** (۱۹۹۸، یان لوکون): اولین CNN عملی، برای خواندنِ ارقامِ چک‌های بانکی — دقیقاً کاربردِ ما!
- **AlexNet** (۲۰۱۲): CNN عمیقی که مسابقهٔ ImageNet را با اختلافِ زیاد برد و انقلابِ یادگیریِ عمیق را آغاز کرد.
- **VGG** (۲۰۱۴): نشان داد که عمقِ بیشتر با کرنل‌های کوچکِ × بهتر است.

- **ResNet** (۲۰۱۵): با «اتصال‌های میان‌بُر» (skip connections) آموزشِ شبکه‌های بسیار عمیق (۵۰ تا ۱۵۰ لایه) را ممکن کرد.

نکتهٔ کلیدی

همه‌شان همان بلوک‌های Conv+ReLU+Pool را دارند که امروز یاد گرفتیم — فقط عمیق‌تر و بزرگ‌تر. وقتی این بلوک‌ها را فهمیدید، فهمِ معماری‌های پیشرفته فقط مسئلهٔ مقیاس است. در هفتهٔ ۱۴ از یک شبکهٔ ازپیش‌آموخته (مثلِ ResNet) برای پروژهٔ خودمان استفاده می‌کنیم.

۴.۵ چرا CNN به GPU و دادهٔ زیاد نیاز دارد

هر تصویر میلیون‌ها ضرب‌وجمع دارد و GPU آن‌ها را به‌صورتِ موازیِ انبوه انجام می‌دهد. شبکه‌های عمیق میلیون‌ها پارامتر دارند و برای برازشِ بدونِ بیش‌برازش به نمونه‌های فراوان نیاز دارند. شبکهٔ ارقامِ ما بسیار کوچک است و در چند ثانیه آموزش می‌بینند؛ شبکه‌های در مقیاسِ ImageNet روزها آموزش می‌بینند.

هشدار / دام

دادهٔ کم + شبکهٔ عمیق = بیش‌برازش. **یادگیریِ انتقالیِ** هفتهٔ آینده دقیقاً همین مشکل را حل می‌کند: به‌جای آموزش از صفر، از شبکه‌ای که قبلاً روی میلیون‌ها تصویر آموزش دیده استفاده می‌کنیم.

۶ کدِ CNN در PyTorch

```
import torch, torch.nn as nn
class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1 = nn.Conv2d(1, 8, 3, padding=1)
        self.c2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(16*2*2, 10)
    def forward(self, x):
        x = self.pool(torch.relu(self.c1(x)))   # 8x8 -> 4x4
        x = self.pool(torch.relu(self.c2(x)))   # 4x4 -> 2x2
        return self.fc(x.flatten(1))            # -> 10 scores
```

کانولوشن → ReLU → Pool، دو بار، و سپس یک طبقه‌بندِ خطی. کلِ خوانندهٔ ارقام همین است.

۷ تمرینِ کلاسی و پاسخ

تمرین

۱) یک لایهٔ کانولوشنی چه نسبتی با کرنلِ سوبلِ هفتهٔ ۳ دارد؟

۲) چرا روی‌هم‌گذاشتنِ کانولوشن‌ها بدونِ ReLU مدلِ قوی‌تری نمی‌دهد؟

۳) CNN ما ۹۷٪ و SVM ۹۹٪ روی ارقام گرفت. آیا این یعنی هاCNN بدترند؟ توضیح دهید.

پاسخ‌ها: (۱) هر دو یک کرنلِ کوچک را روی تصویر کانولوشن می‌کنند؛ اعدادِ سوبل ثابت‌اند، اعدادِ CNN آموخته می‌شوند. (۲) ترکیبِ عملیاتِ خطی باز هم خطی است — غیرخطی‌بودنِ ReLU همان چیزی است که قدرت می‌افزاید. (۳) نه — روی داده‌های ساده و تمیز، یادگیریِ ماشینِ کلاسیک از قبل نزدیک‌به‌کامل است؛ هاCNN روی داده‌های سخت، متنوع و بزرگ‌مقیاس و با یادگیریِ انتقالی برنده می‌شوند.

۸ فهرستِ عیب‌یابیِ عملیِ آموزش

وقتی CNN خودتان را آموزش می‌دهید و کار نمی‌کند، این فهرست را به‌ترتیب چک کنید:

- **زیان اصلاً حرکت نمی‌کند؟** نرخِ یادگیری احتمالاً خیلی کوچک است، یا داده نرمال نشده (مقادیر را به بازهٔ $[۰, ۱]$ ببرید).
- **زیان منفجر شد (NaN)؟** نرخِ یادگیری خیلی بزرگ است؛ ده برابر کوچکش کنید.
- **دقتِ آموزش بالا اما آزمون پایین؟** بیش‌برازش؛ dropout، augmentation یا دادهٔ بیشتر اضافه کنید.
- **هر دو دقت پایین؟** کم‌برازش؛ شبکه را بزرگ‌تر یا آموزش را طولانی‌تر کنید.
- **همه‌چیز درست به‌نظر می‌رسد اما جواب بد است؟** برچسب‌ها را چک کنید — اشتباهِ رایج، ناهم‌ترازیِ تصویر و برچسب است.

هشدار / دام

پیش از هر آموزشِ بزرگ، شبکه را روی یک دستهٔ کوچک (مثلاً ۱۰ نمونه) امتحان کنید. اگر شبکه نتواند حتی ۱۰ نمونه را حفظ کند (به دقتِ ۱۰۰٪ روی همان‌ها برسد)، یک باگِ اساسی در کد دارید. این «آزمونِ بیش‌برازشِ عمدی» سریع‌ترین راهِ یافتنِ باگ است.

۹ جمع‌بندی

در این جلسه یاد گرفتیم که CNN همان کانولوشنِ هفتهٔ ۳ است اما با فیلترهای آموخته، به‌همراهِ pooling ،ReLU و یک طبقه‌بند؛ آموزش با گذرِ روبه‌جلو، محاسبهٔ زیان، پس‌انتشار و گرادیان کاهشی انجام می‌شود؛ منحنی‌های آموزش، نقشِ دسته و epoch و نرخِ یادگیری را نشان می‌دهند؛ و روی داده‌های ساده کلاسیک هم عالی است اما هاCNN روی مسائلِ سخت جلو می‌افتند. مهم‌تر از همه، دیدیم که دادهٔ کم CNN را به بیش‌برازش می‌کشاند — و همین، انگیزهٔ **یادگیریِ انتقالیِ** هفتهٔ ۱۴ و تکمیلِ پروژهٔ DocScan است.